# Universidad Nacional del Comahue

Facultad de Informática

Diseño de Compiladores e Interpretes

2025

# Trabajos Prácticos de Laboratorio

**Manuel Felipe Triñanes**

*manuel.trinanes@est.fi.uncoma.edu.ar*

FAI-2738

**Bianca Antonella Torres**

*bianca.torres@est.fi.uncoma.edu.ar*

FAI-2991

# Índice


**1. Trabajo de Laboratorio N1: Definición Mini Pascal** **4**
1.1. Descripción del problema . . . . . 4
1.2. Desarrollo . . . . . 4
1.2.1. Estrategia y Metalenguaje . . . . . 4
1.3. Definición de la Gramática . . . . . 5
1.4. Problemas Encontrados . . . . . 7
1.5. Conclusiones . . . . . 7

**2. Trabajo de Laboratorio N2** **8**
2.1. Descripción del problema . . . . . 8
2.2. Desarrollo . . . . . 8
2.2.1. Alfabeto . . . . . 8
2.2.2. Lexemas . . . . . 9
2.2.3. Expresiones Regulares . . . . . 10
2.2.4. Automatas . . . . . 11
2.3. Conclusiones . . . . . 13

**3. Trabajo de Laboratorio N3: Analizador Léxico** **14**
3.1. Descripción del Producto y Herramientas Utilizadas para su Desarrollo . . . . . 14
3.2. Manual de Uso . . . . . 14
3.3. Ejemplos de ejecuciones . . . . . 15
3.4. Problemas Encontrados . . . . . 17
3.5. Conclusiones . . . . . 17

**4. Trabajo de Laboratorio N4: LL1 y Pseudo-Sintáctico** **19**
4.1. Factorización y Recursión a Izquierda . . . . . 19
4.1.1. Factorización a Izquierda . . . . . 19
4.1.2. Recusrión a Izquierda . . . . . 20
4.2. Eliminación de BFN Extendido . . . . . 20
4.3. Gramáticas LL(1) . . . . . 23
4.4. Analizador Sintáctico (pesudocodigo) . . . . . 25

**5. Trabajo de Laboratorio N5: Analizador Sintáctico** **29**
5.1. Descripción del Producto y Herramientas Utilizadas para su Desarrollo . . . . . 29
5.2. Manual de Uso . . . . . 29
5.3. Ejemplos de ejecuciones . . . . . 30
5.4. Problemas Encontrados . . . . . 31
5.5. Conclusiones . . . . . 32

**6. Trabajo Practico N6: Analizador Semántico** **33**
6.1. Tabla de Símbolos . . . . . 33
6.2. Implementacion . . . . . 34
6.2.1. La Tabla de Símbolos (`TablaSimbolos`) . . . . . 34
6.2.2. El Analizador Semántico (`AnalizadorSemantico`) . . . . . 35
6.2.3. Integración con el Analizador Sintáctico . . . . . 35
6.2.4. Manejo de Errores Semánticos . . . . . 36

**7. Trabajo de Laboratorio N7: Generador de Código Intermedio (MEPA)** **37**
7.1. El Árbol de Sintaxis Abstracta (AST) . . . . . 37
7.1.1. Taxonomía de Nodos . . . . . 37
7.1.2. Enriquecimiento Semántico del AST . . . . . 38
7.2. Diseño del Generador de Código MEPA . . . . . 38
7.2.1. Fase 1: Pre-Cálculo de Nivel Léxico y Desplazamientos (Offsets) . . . . . 38
7.2.2. Fase 2: Generación de Código (Visitor Pattern) . . . . . 39
7.3. Código de Inicialización del Programa . . . . . 40
7.4. Manual de Uso . . . . . 40

**Referencias** **42**

# 1. Trabajo de Laboratorio N1: Definición Mini Pascal

## 1.1. Descripción del problema

Este informe busca definir, mediante la notación BNF extendida (EBNF), un subconjunto simplificado del lenguaje Pascal que incluye los siguientes elementos:

- Declaración de variables y subprogramas (procedimientos y funciones con paso de parámetros por valor).
- Tipos de datos básicos: *integer* y *boolean*.
- Constantes predefinidas: `true` y `false`.
- Sentencias de control: asignación, alternativa, repetitiva y compuesta.
- Procedimientos de entrada/salida: `read` y `write` para un solo valor entero.
- Expresiones aritméticas con operadores $+$, $-$ (unario y binario), $^*$, $/$ y paréntesis.
- Expresiones booleanas con operadores lógicos (`AND`, `OR`, `NOT`) y de comparación ($>$, $<$, $=$, $\leq$, $\geq$, $<>$).

El objetivo es establecer una gramática formal que describa su sintaxis de manera clara y útil para análisis o implementación.

## 1.2. Desarrollo

### 1.2.1. Estrategia y Metalenguaje

La estrategia a ser utilizada es el ahcer uso del metalenguaje *Extended Backus–Naur Form* (BNFE), o en español, *Notación de Backus-Naur Extendido* el cual es utilizado para expresar gramáticas libres de contexto. En este metalenguaje el símbolo:

- $< >$ $\rightarrow$ no terminal.
- **terminal** (negrita).
- $|$ $\rightarrow$ alternativa.
- $::=$ $\rightarrow$ asignación.
- ( ) $\rightarrow$ agrupación.
- [ ] $\rightarrow$ item optativo.
- { } $\rightarrow$ repetitiva de 0 o más veces.

## 1.3. Definición de la Gramática

### Programas y Bloques

$$< programa > ::= program < identificador > ; < bloque > .$$
$$< bloque > ::= [< declaraciones\ de\ variables >]$$
$$[< declaracion\ de\ subrutinas >]$$
$$< sentencia\ compuesta >$$

### Declaracion de Variables

$$< parte\ de\ declaraciones\ de\ variables > ::= var < declaracion\ de\ variables >$$
$$\{; < declaracion\ de\ variables >\};$$
$$< declaracion\ de\ variables > ::= < lista\ identificadores > : < tipo >$$
$$< lista\ identificadores > ::= < identificador > \{, < identificador >\}$$
$$< identificador > ::= < letra > \{< letra > | < digito >\}$$
$$< letra > ::= a|b|c|d|ef|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z$$
$$< digito > ::= 0|1|2|3|4|5|6|7|8|9$$

### Declaracion de Sub-programas

$$< \text{declaracion de subrutina} > ::= \{< \text{declaracion de procedimiento} > ; |$$
$$< \text{declaracion de funcion} > ;\}$$
$$< \text{declaracion de procedimiento} > ::= procedure < \text{identificador} >$$
$$[< \text{parámetros formales} >]; < \text{bloque} >$$
$$< \text{declaracion de funcion} > ::= function < \text{identificador} >$$
$$[< \text{parámetros formales} >] : < tipo > ; < \text{bloque} >$$
$$< \text{parámetros formales} > ::= ( < \text{seccion de parámetros formales} >$$
$$\{; < \text{seccion de parámetros formales} >\})$$
$$< \text{seccion de parámetros formales} > ::= < \text{lista de identificadores} > : < \text{tipo} >$$

### Tipos de datos simples

$$< tipo > ::= integer \mid boolean$$

### Sentencias

$$
\begin{aligned}
< \text{sentencia compuesta} > ::= \; & \textit{begin} < \text{sentencia} > \\
& \{; < \text{sentencia} >\} \; \textit{end} \\
< \text{sentencia} > ::= \; & < \text{asignacion} > \\
& \mid < \text{llamada a procedimiento} > \\
& \mid < \text{sentencia compuesta} > \\
& \mid < \text{sentencia condicional} > \\
& \mid < \text{sentencia repetitiva} > \\
& \mid < \text{sentencia lectura} > \\
& \mid < \text{sentencia escritura} > \\
< \text{asignacion} > ::= \; & < \text{variable} > \; := \; < \text{expresion} > \\
< \text{sentencia condicional} > ::= \; & \textit{if} < \text{expresion} > \textit{then} < \text{sentencia} > \\
& [\textit{else} < \text{sentencia} >] \\
< \text{sentencia repetitiva} > ::= \; & \textit{while} < \text{expresion} > \textit{do} < \text{sentencia} > \\
< \text{sentencia lectura} > ::= \; & \textit{read}( < \text{identificador} > ) \\
< \text{sentencia escritura} > ::= \; & \textit{write}( \; < \text{identificador} > \; )
\end{aligned}
$$

**Expresiones**

$$
\begin{aligned}
< \text{lista de expresiones} > ::= \; & < \text{expresion} > \{, < \text{expresion} >\} \\
< \text{expresion} > ::= \; & < \text{expresion simple} > \\
& [< \text{relacion} >< \text{expresion simple} >] \\
< \text{relacion} > ::= \; & = \mid <> \mid < \mid > \mid <= \mid >= \\
< \text{expresion simple} > ::= \; & [+|-] < \text{termino} > \\
& \{(+|-|or) < \text{termino} >\} \\
< \text{término} > ::= \; & < \text{factor} > \\
& \{(*|div|and) < \text{factor} >\} \\
< \text{factor} > ::= \; & < \text{identificador} > \\
& |numero \\
& \mid < \text{llamada a funcion} > \\
& |(< \text{expresion} >) \\
& |not < \text{factor} > \\
& |true \\
& |false \\
< \text{llamada a funcion} > ::= \; & < \text{identificador} > [( < \text{lista de expresiones} > )]
\end{aligned}
$$

## 1.4. Problemas Encontrados

Como principal problema encontrado fue la falta de conocimiento sobre el lenguaje Pascal, ya que este solo lo hemos utilizado en un Trabajo Practico no obligatorio en Principios de Lenguajes de Programación. Y, ademas, la falta de practica para la declaración de BNF.

## 1.5. Conclusiones

Este trabajo se centró en la adaptación de una gramática simplificada de Pascal basada en la propuesta (Kowaltowski, 1983, Capitulo 5), con el objetivo de ajustarla a un conjunto específico de requerimientos. Durante el proceso, se tomó la decisión de priorizar la simplicidad sobre la completitud, lo que dio lugar a una versión más manejable pero con una cobertura limitada del lenguaje original. Si bien esto restringe algunas de sus capacidades, también facilita su comprensión y aplicación en entornos educativos o de análisis de estructuras básicas del lenguaje. Un ejemplo claro de esto es la capacidad del lenguaje Pascal de poder pasar como parámetro de una función otra función, lo cual no es posible con la gramática presentada en este trabajo.

# 2. Trabajo de Laboratorio N2

## 2.1. Descripción del problema

El problema consiste en analizar un subconjunto del lenguaje Pascal con comentarios delimitados por { y }, junto con características adicionales mencionadas en el Trabajo de Laboratorio No. 1 (1.1). El propósito es formalizar su estructura léxica para facilitar la creación de un analizador léxico que procese correctamente los comentarios y otros elementos definidos.

Para ello, se debe identificar el alfabeto de entrada, que incluye los símbolos básicos del lenguaje necesarios para los comentarios y otros componentes léxicos. Luego, es necesario determinar los lexemas, secuencias de caracteres con significado, y clasificarlos en tokens que representen categorías léxicas, como los comentarios. Además, se requiere describir los patrones de estos lexemas con expresiones regulares que formalicen las secuencias válidas. Por último, se solicita diseñar un autómata finito con un diagrama de transición de estados para reconocer los tokens, modelando el proceso de análisis léxico. Este enfoque proporciona una base sólida para procesar eficientemente las construcciones del subconjunto de Pascal especificado.

## 2.2. Desarrollo

### 2.2.1. Alfabeto

El alfabeto de entrada corresponiente a nuestro subconjunto de Pascal simplificado, descripto en 1.1 sumado a '{' y '}' para los comentarios es:

$$\Sigma = \{\mathrm{A}, \mathrm{B}, \ldots, \mathrm{Z}, \mathrm{a}, \mathrm{b}, \ldots, \mathrm{z}, 0, 1, \ldots, 9, +, -, *, /, (, ), >, <, =, :, ;, \{, \}, ,, .\}$$

### 2.2.2. Lexemas

| Lexema | Nombre Token | Valor Atributo |
|---|---|---|
| Cualquier Identificador | **id** | Puntero a la entrada en TS |
| Cualquier Numero | **numero** | Puntero a la entrada en TS |
| program | **program** | |
| var | **var** | |
| procedure | **procedure** | |
| fucntion | **function** | |
| integer | **integer** | |
| boolean | **boolean** | |
| begin | **begin** | |
| end | **end** | |
| if | **if** | |
| then | **then** | |
| else | **else** | |
| while | **while** | |
| do | **do** | |
| or | **oper_logico** | or |
| and | **oper_logico** | and |
| not | **oper_logico** | not |
| : | **asignacion_de_tipo** | |
| := | **asignacion** | |
| ; | **punto_coma** | |
| , | **coma** | |
| . | **punto** | |
| ( | **parentesis_izq** | |
| ) | **parentesis_der** | |
| + | **oper_aritmetico** | suma |
| - | **oper_aritmetico** | resta |
| * | **oper_aritmetico** | mult |
| / | **oper_aritmetico** | div |
| = | **oper_relacional** | igual |
| < | **oper_relacional** | menor |
| <= | **oper_relacional** | menor_igual |
| > | **oper_relacional** | mayor |
| >= | **oper_relacional** | mayor_igual |
| <> | **oper_relacional** | distinto |

Cuadro 1: Tabla de lexemas, tokens y valores/atributos.

### 2.2.3. Expresiones Regulares

| Token | Patrón | Lexema de ejemplo |
|---|---|---|
| program | program | program |
| id | $(a-z \cup A-Z)(a-z \cup A-Z \cup 0-9)^*$ | holaMundo10 |
| var | var | var |
| asignacion_tipo | : | : |
| asignacion | := | := |
| punto_coma | ; | ; |
| coma | , | , |
| punto | . | . |
| parentesis_izq | ( | ( |
| parentesis_der | ) | ) |
| procedure | procedure | procedure |
| function | function | function |
| integer | integer | integer |
| boolean | boolean | boolean |
| begin | begin | begin |
| end | end | end |
| if | if | if |
| then | then | then |
| else | else | else |
| while | while | while |
| do | do | do |
| oper_logico | OR \| AND \| NOT | OR |
| numero | $(0 - 9)^+$ | 43 |
| oper_aritmetico | + \| - \| * \| / | + |
| oper_relacional | = \| <\| <= \| >\| >= \| <> | < |
| | $\{(\Sigma - \{\})^*\}$ | {esto es un comentario} |

Cuadro 2: Tokens, patrones y ejemplos de lexemas.

### 2.2.4. Automatas

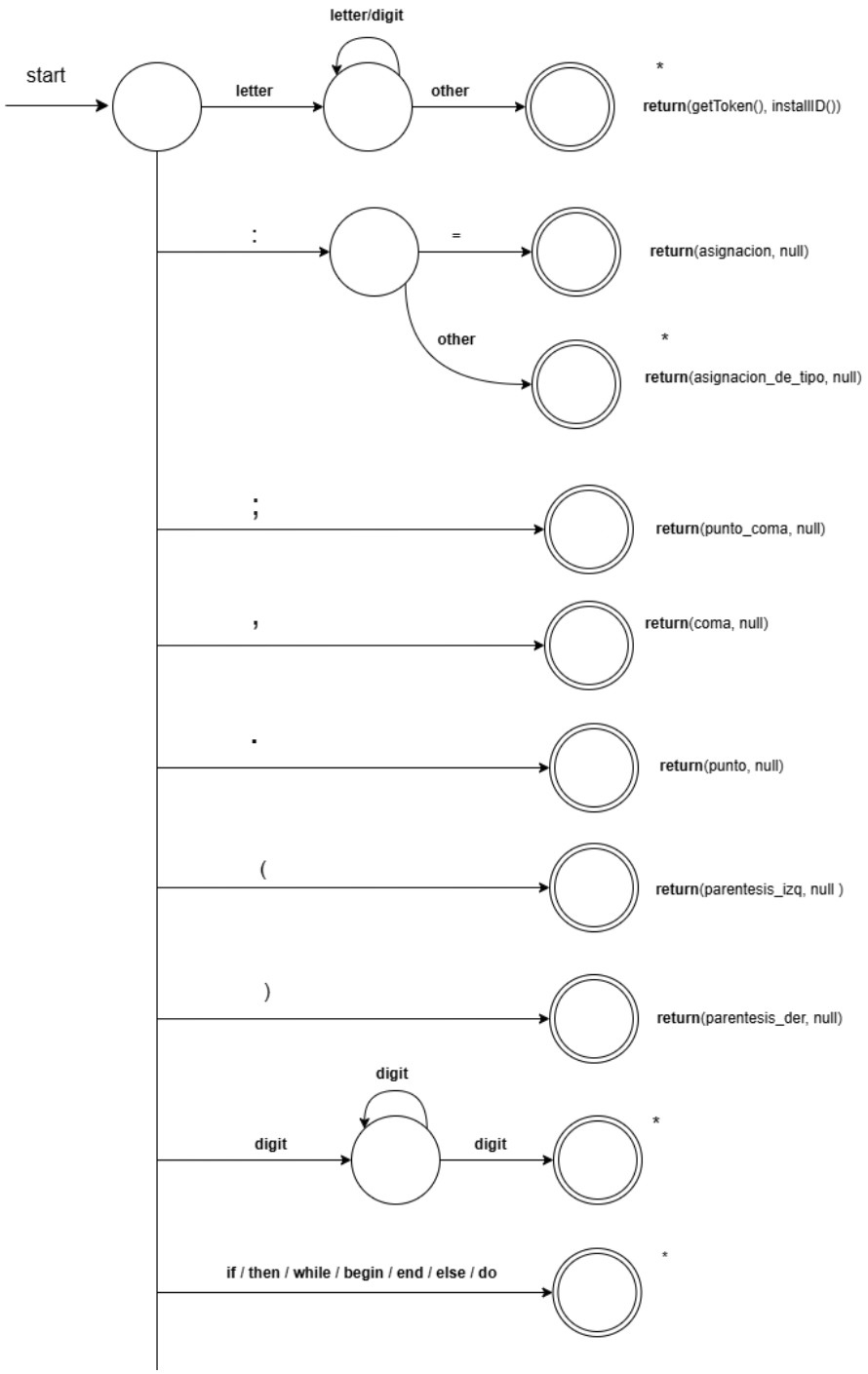


Figura 1: Automata

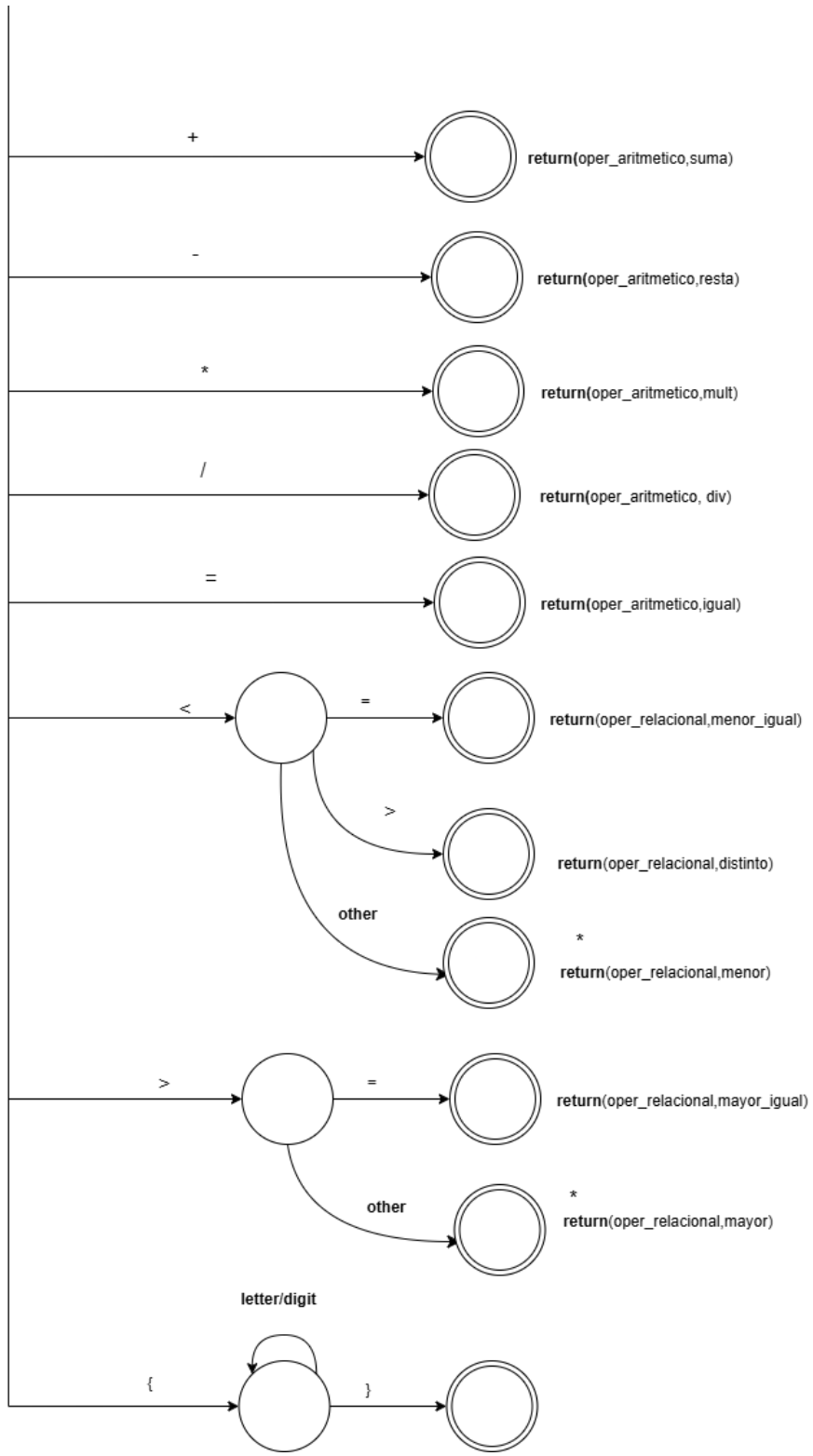


Figura 2: Automata *(continuación)*

## Aclaraciones

Para optimizar el diseño del autómata, en lugar de construir un autómata específico para cada palabra reservada de Pascal, como `else`, se optó por representar estas palabras mediante un único arco etiquetado directamente con la palabra completa (por ejemplo, `else`). Esto simplifica el autómata, reduciendo la cantidad de estados y transiciones necesarios. La figura siguiente ilustra el autómata específico para `else` como referencia:

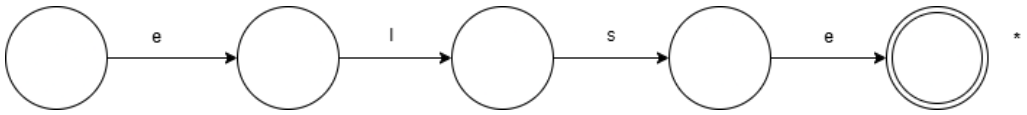


Figura 3: Autómata para la palabra reservada `else`

Esta simplificación se aplicó de manera generalizada a todas las palabras reservadas, integrándolas en un solo arco dentro del autómata global separadas por un '/' (ej: *else* / *if* / ...), lo que mejora la eficiencia y claridad del diseño.

## 2.3. Conclusiones

En conclusión, se definieron los elementos fundamentales del lenguaje, como el alfabeto de entrada, los lexemas, los tokens y sus respectivos patrones. A partir de esta especificación, se diseñó un autómata finito que permite identificar correctamente los distintos tokens del subconjunto de Pascal.

# 3. Trabajo de Laboratorio N3: Analizador Léxico

## 3.1. Descripción del Producto y Herramientas Utilizadas para su Desarrollo

El producto desarrollado es un analizador léxico diseñado para reconocer los tokens especificados en la Sección 2. Este analizador toma como entrada un archivo de texto fuente y lo procesa línea por línea, identificando palabras reservadas, operadores aritméticos, operadores lógicos, símbolos de puntuación, identificadores y números. El analizador genera una lista de tokens con sus correspondientes categorías, facilitando la posterior etapa de análisis sintáctico o semántico.

Para su desarrollo, se utilizó el lenguaje de programación Python debido a su flexibilidad y la facilidad para manejar cadenas de texto. En particular, se emplearon las siguientes herramientas y técnicas:

- **Python 3.x**: El lenguaje de programación principal utilizado para desarrollar el analizador léxico. Python es adecuado para este tipo de tareas debido a su sintaxis simple y sus potentes bibliotecas.
- **Editor de texto**: Se utilizó un editor de texto como Visual Studio Code para escribir y organizar el código.
- **Autómata**: El analizador está basado en el autómata desarrollado en la sección 2.2.4, el cual define los estados y transiciones necesarias para reconocer los diferentes tokens del lenguaje.
- **Archivos de texto**: El programa toma como entrada un archivo de texto (.txt), lo lee y procesa su contenido.

## 3.2. Manual de Uso

Para ejecutar el analizador léxico, siga los siguientes pasos:

1. **Preparación del entorno de trabajo:**
   - Asegúrese de tener instalado Python 3.x en su sistema. Puede descargarlo desde el sitio oficial: `https://www.python.org/downloads/`.
2. **Cargar el archivo de entrada:**
   - Coloque el archivo de texto que contiene el código fuente que desea analizar en el mismo directorio que el programa o proporcione la ruta completa al archivo.
   - Asegúrese de que el archivo de entrada esté en formato .txt y contenga el código del lenguaje a analizar.

- En la terminal, ejecute el script Python con uno de los siguientes comandos, según cómo esté configurado su sistema:

  ```
  python analizador_lexico.py entrada.txt
  ```

  o bien:

  ```
  py analizador_lexico.py entrada.txt
  ```

- Asegúrese de estar ubicado en el mismo directorio que el archivo `analizador_lexico.py` y que el archivo `entrada.txt` exista.

3. **Ver los resultados:**

   - Después de ejecutar el programa, los tokens se mostrarán en la terminal en el siguiente formato:

     ```
     (lin. 001, col. 001)  program        ->  program
     (lin. 001, col. 009)  var            ->  var
     (lin. 001, col. 013)  nombreVariable  ->  id
     (lin. 002, col. 001)  asignacion_de_tipo ->  :
     (lin. 002, col. 003)  asignacion     ->  :=
     ```

## 3.3. Ejemplos de ejecuciones

**Entrada:**

```
program ejemplo;
var x: integer;
begin
    x := 42;
end.
```

**Salida:**

```
(lin. 001, col. 009)  program        ->  program
(lin. 001, col. 017)  ejemplo        ->  id
(lin. 001, col. 017)  ;              ->  punto_coma
(lin. 002, col. 008)  var            ->  var
(lin. 002, col. 010)  x              ->  id
(lin. 002, col. 011)  :              ->  asignacion_de_tipo
(lin. 002, col. 019)  integer        ->  integer
(lin. 002, col. 019)  ;              ->  punto_coma
(lin. 003, col. 010)  begin          ->  begin
(lin. 004, col. 010)  x              ->  id
(lin. 004, col. 012)  :=             ->  asignacion
(lin. 004, col. 016)  42             ->  numero
(lin. 004, col. 016)  ;              ->  punto_coma
(lin. 005, col. 008)  end            ->  end
(lin. 005, col. 008)  .              ->  punto
```

Figura 4: Salida ejecución ejemplo sin errores

**Entrada:**

```
program ejemplo;
var x: integer;
{chequea si 42 es mayor a 50@}
begin @
    x := 42;
    if x < 50 then
        x := x + 1;
    else
        x := 0;
end.
```

**Salida:**

```
(lin. 001, col. 008)  program        ->  program
(lin. 001, col. 016)  ejemplo        ->  id
(lin. 001, col. 016)  ;              ->  punto_coma
(lin. 002, col. 004)  var            ->  var
(lin. 002, col. 006)  x              ->  id
(lin. 002, col. 007)  :              ->  asignacion_de_tipo
(lin. 002, col. 015)  integer        ->  integer
(lin. 002, col. 015)  ;              ->  punto_coma
(lin. 004, col. 006)  begin          ->  begin
(lin. 004, col. 007)  @              ->  ERROR
(lin. 005, col. 006)  x              ->  id
(lin. 005, col. 008)  :=             ->  asignacion
(lin. 005, col. 012)  42             ->  numero
(lin. 005, col. 012)  ;              ->  punto_coma
(lin. 006, col. 007)  if             ->  if
(lin. 006, col. 009)  x              ->  id
(lin. 006, col. 011)  <              ->  oper_relacional, menor
(lin. 006, col. 014)  50             ->  numero
(lin. 006, col. 019)  then           ->  then
(lin. 007, col. 010)  x              ->  id
(lin. 007, col. 012)  :=             ->  asignacion
(lin. 007, col. 015)  x              ->  id
(lin. 007, col. 016)  +              ->  oper_aritmetico, suma
(lin. 007, col. 019)  1              ->  numero
(lin. 007, col. 019)  ;              ->  punto_coma
(lin. 008, col. 009)  else           ->  else
(lin. 009, col. 010)  x              ->  id
(lin. 009, col. 012)  :=             ->  asignacion
(lin. 009, col. 015)  0              ->  numero
(lin. 009, col. 015)  ;              ->  punto_coma
(lin. 010, col. 004)  end            ->  end
(lin. 010, col. 004)  .              ->  punto
```

Figura 5: Salida ejecución ejemplo con errores

Si se encuentra con algún error de token no esperado, el analizador mostrará un mensaje de error, indicando el carácter problemático y su posición en el archivo. Tal como se puede ver en la linea 4 col 7.

## 3.4. Problemas Encontrados

Un problema identificado durante el desarrollo del analizador léxico está relacionado con el manejo del último lexema, especialmente cuando este se encuentra al final del programa de entrada. En este caso, luego de procesar el último carácter del texto, la cabeza lectora avanza una posición más, lo que provoca que intente acceder a un índice fuera del rango de la cadena. Esto genera un error de desbordamiento de índice.

Para solucionar este inconveniente, se optó por agregar un carácter de espacio en blanco al final del programa de entrada, de modo que el analizador siempre pueda identificar el final del texto y evitar el intento de lectura más allá de los límites válidos.

## 3.5. Conclusiones

Se logró desarrollar el analizador léxico en Python tomando como base el autómata definido previamente. Se detalló su comportamiento general, se presentaron ejemplos de ejecución para

ilustrar su salida y se documentó el principal inconveniente detectado durante el desarrollo, junto con su solución.

# 4. Trabajo de Laboratorio N4: LL1 y Pseudo-Sintáctico

## 4.1. Factorización y Recursión a Izquierda

### 4.1.1. Factorización a Izquierda

La factorización a izquierda (o left factoring en inglés) es una técnica descrita por Aho, Sethi, y Ullman (1986) la cual se utiliza para transformar una gramática ambigua o no adecuada para ciertos tipos de parsers (como los descendentes predictivos o LL) en una equivalente que sea más fácil de procesar.

La factorización a izquierda se aplica cuando varias producciones de un no terminal comparten un prefijo común. Esto puede causar problemas en un analizador descendente predictivo, ya que el parser no puede decidir cuál producción elegir basándose únicamente en el símbolo lookahead. La técnica consiste en extraer el prefijo común y reestructurar la gramática para eliminar la ambigüedad en la elección.

Por ejemplo la siguiente gramática:

$$S \rightarrow \text{if } E \text{ then } S_1 \mid \text{if } E \text{ then } S_1 \text{ else } S_2$$

El prefijo común es `if` $E$ `then`. Aplicando factorización a izquierda, se obtiene:

$$S \rightarrow \text{if } E \text{ then } S'$$

$$S' \rightarrow S_1 \mid S_1 \text{ else } S_2$$

Luego de analizar la gramática actualizada, definida en la sección 4.2, se encontró en dos de las reglas de la gramática la necesidad de factorización a izquierda, siendo estas la siguiente:

**Expresiones**

$$\begin{aligned}
< \text{factor} > ::= & < \text{identificador} > \\
& \mid < \text{llamada a funcion} > \\
< \text{llamada a funcion} > ::= & < \text{identificador} >< \text{parte parametros actuales} > \\
< \text{parte parametros actuales} > ::= & \ ( < \text{lista de expresiones} > ) \\
& \mid ( ) \\
& \mid \lambda
\end{aligned}$$

**Expresiones**

$$\begin{aligned}
< \text{factor} > ::= & < \text{identificador} >< \text{factor identificador} > \\
< \text{factor identificador} > ::= & < \text{parte parametros actuales} > \mid \lambda \\
< \text{parte parametros actuales} > ::= & \ ( < \text{resto parametros actuales} > \mid \lambda \\
< \text{resto parametros actuales} > ::= & \ ) \mid < \text{lista de expresiones} > )
\end{aligned}$$

**Sentencias**

$$
\begin{aligned}
< \text{sentencia} > ::= \ & < \text{asignacion} > \\
| \ & < \text{llamada a procedimiento} > \\
< \text{asignacion} > ::= \ & < \text{identificador} > \ := \ < \text{expresion} > \\
< \text{llamada a procedimiento} > ::= \ & < \text{identificador} > < \text{parte parametros actuales} >
\end{aligned}
$$

**Sentencias**

$$
\begin{aligned}
< \text{sentencia} > ::= \ & < \text{identificador} >< \text{sentencia ident} > \\
< \text{sentencia ident} > ::= \ & := \ < \text{expresion} > \ | \ < \text{parte parametros actuales} >
\end{aligned}
$$

### 4.1.2. Recusrión a Izquierda

Es posible que un analizador descendente recursivo entre en un bucle infinito. Surge un problema con producciones "recursivas a izquierda" como

$$\texttt{stmt} \rightarrow \texttt{stmt}; \texttt{cmd}$$

donde el símbolo más a la izquierda del cuerpo es el mismo que el no terminal a la cabeza de la producción en uno o más pasos. En este ejemplo, el cuerpo comienza con `stmt`, entonces se llama al procedimiento `stmt` recursivamente. Como el símbolo *lookahead* solo cambia cuando un terminal en el cuerpo realiza *match*, no habrá cambios en la entrada entre las llamadas recursivas a `stmt`, y por lo tanto se entrará en un bucle infinito (Aho y cols., 1986).

Luego de analizar la gramática definida en la sección 1.3, se concluyó que la misma no presenta recursión a izquierda.

## 4.2. Eliminación de BFN Extendido

Ademas de la factorización a izquierda mencionada anteriormente, se remueven los símbolos pertenecientes a BNF Extendido, siendo estos los parentesis ( ) para agrupación, [ ] para opcionales y { } para repetitivas. Como resultado se obtiene esta segunda versión a la gramática propuesta en la sección 1.3:

## Programas y Bloques

$$
\begin{aligned}
< programa > ::= & \ program < identificador > ; < bloque > . \\
< bloque > ::= & \ < parte\ declaraciones\ variables > \\
& \ < parte\ declaraciones\ subrutinas > \\
& \ < sentencia\ compuesta > \\
< bloque > ::= & \ < parte\ declaraciones\ variables > \\
& \ < sentencia\ compuesta > \\
< bloque > ::= & \ < parte\ declaraciones\ subrutinas > \\
& \ < sentencia\ compuesta > \\
< bloque > ::= & \ < sentencia\ compuesta > \\
< parte\ declaraciones\ variables > ::= & \ var < declaracion\ de\ variables >< mas\ declaraciones > ; \\
< mas\ declaraciones > ::= & \ ; < declaracion\ de\ variables >< mas\ declaraciones > \\
& \ | \ \lambda \\
< parte\ declaraciones\ subrutinas > ::= & \ < declaracion\ de\ subrutina >< mas\ subrutinas > \\
< mas\ subrutinas > ::= & \ ; < declaracion\ de\ subrutina >< mas\ subrutinas > \\
& \ | \ \lambda
\end{aligned}
$$

## Declaracion de Variables

$$
\begin{aligned}
< declaracion\ de\ variables > ::= & \ < lista\ identificadores > \ : \ < tipo > \\
< lista\ identificadores > ::= & \ < identificador >< mas\ identificadores > \\
< mas\ identificadores > ::= & \ , < identificador >< mas\ identificadores > \\
& \ | \ \lambda \\
< identificador > ::= & \ < letra >< resto\ identificador > \\
< resto\ identificador > ::= & \ < letra >< resto\ identificador > \\
& \ | \ < digito >< resto\ identificador > \\
& \ | \ \lambda \\
< letra > ::= & \ a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z \\
< digito > ::= & \ 0|1|2|3|4|5|6|7|8|9
\end{aligned}
$$

## Declaracion de Sub-programas

$$
\begin{aligned}
< \text{declaracion de subrutina} > ::= & \ < \text{declaracion de procedimiento} > \\
& \ | \ < \text{declaracion de funcion} > \\
< \text{declaracion de procedimiento} > ::= & \ procedure < \text{identificador} > \\
& \ < \text{parte parametros formales} > ; < \text{bloque} > \\
< \text{declaracion de funcion} > ::= & \ function < \text{identificador} > \\
& \ < \text{parte parametros formales} > \ : \ < tipo > ; < \text{bloque} >
\end{aligned}
$$

$$
\begin{aligned}
< \text{parte parametros formales} > ::= & \ ( < \text{seccion de parametros formales} > \\
& \quad < \text{mas secciones parametros} > ) \\
& \mid \lambda \\
< \text{mas secciones parametros} > ::= & \ ; < \text{seccion de parametros formales} > \\
& \quad < \text{mas secciones parametros} > \\
& \mid \lambda \\
< \text{seccion de parametros formales} > ::= & \ < \text{lista de identificadores} > : < \text{tipo} >
\end{aligned}
$$

### Tipos de datos simples

$$< tipo > ::= integer \mid boolean$$

### Sentencias

$$
\begin{aligned}
< \text{sentencia compuesta} > ::= & \ begin < \text{sentencia} >< \text{mas sentencias} > \ end \\
< \text{mas sentencias} > ::= & \ ; < \text{sentencia} >< \text{mas sentencias} > \\
& \mid \lambda \\
< \text{sentencia} > ::= & \ < \text{asignacion} > \\
& \mid < \text{llamada a procedimiento} > \\
& \mid < \text{sentencia compuesta} > \\
& \mid < \text{sentencia condicional} > \\
& \mid < \text{sentencia repetitiva} > \\
& \mid < \text{sentencia lectura} > \\
& \mid < \text{sentencia escritura} > \\
< \text{asignacion} > ::= & \ < \text{identificador} > \ := \ < \text{expresion} > \\
< \text{llamada a procedimiento} > ::= & \ < \text{identificador} > < \text{parte parametros actuales} > \\
< \text{sentencia condicional} > ::= & \ if < \text{expresion} > then < \text{sentencia} >< \text{parte else} > \\
< \text{parte else} > ::= & \ else < \text{sentencia} > \\
& \mid \lambda \\
< \text{sentencia repetitiva} > ::= & \ while < \text{expresion} > do < \text{sentencia} > \\
< \text{sentencia lectura} > ::= & \ read( < \text{identificador} > ) \\
< \text{sentencia escritura} > ::= & \ write( \ < \text{identificador} > \ )
\end{aligned}
$$

### Expresiones

$$< \text{lista de expresiones} > ::= \ < \text{expresion} >< \text{mas expresiones} >$$

$$
\begin{aligned}
< \text{mas expresiones} > ::= & \ , < \text{expresion} >< \text{mas expresiones} > \\
& \mid \lambda \\
< \text{expresion} > ::= & \ < \text{expresion simple} >< \text{parte relacion} > \\
< \text{parte relacion} > ::= & \ < \text{relacion} >< \text{expresion simple} > \\
& \mid \lambda \\
< \text{relacion} > ::= & = \mid <> \mid < \mid > \mid <= \mid >= \\
< \text{expresion simple} > ::= & \ < \text{signo} >< \text{termino} >< \text{resto expresion simple} > \\
& \mid \ < \text{termino} >< \text{resto expresion simple} > \\
< \text{signo} > ::= & + \mid - \\
< \text{resto expresion simple} > ::= & \ < \text{op aditivo} >< \text{termino} >< \text{resto expresion simple} > \\
& \mid \lambda \\
< \text{op aditivo} > ::= & + \mid - \mid or \\
< \text{término} > ::= & \ < \text{factor}>< \text{resto termino} > \\
< \text{resto termino} > ::= & \ < \text{op multiplicativo} >< \text{factor} >< \text{resto termino} > \\
& \mid \lambda \\
< \text{op multiplicativo} > ::= & * \mid div \mid and \\
< \text{factor} > ::= & \ < \text{identificador} >< \text{factor identificador} > \\
& \mid numero \\
& \mid \ < \text{llamada a funcion} > \\
& \mid ( < \text{expresion} > ) \\
& \mid not < \text{factor} > \\
& \mid true \\
& \mid false \\
< \text{factor identificador} > ::= & < \text{parte parametros actuales} > \mid \lambda \\
< \text{parte parametros actuales} > ::= & ( < \text{resto parametros actuales} > \ \mid \lambda \\
< \text{resto parametros actuales} > ::= & ) \mid \ < \text{lista de expresiones} > )
\end{aligned}
$$

## 4.3. Gramáticas LL(1)

Existe una clase de gramáticas conocida como LL(1). La primera "L" en el nombre se debe a que se lee la entrada de izquierda a derecha, la segunda "L" a que se produce una derivación a izquierda, y el "1" a que se usa un símbolo de entrada de *lookahead* para determinar qué acción se tomará en cada paso.

Una gramática $G$ es LL(1) si y solo si cuando existen dos producciones de la forma $A \rightarrow \alpha \mid \beta$, se cumplen las siguientes condiciones:

1. Primeros($\alpha$) y Primeros($\beta$) son disjuntos.
2. Si $\lambda \in$ Primeros($\beta$), entonces Primeros($\alpha$) y Siguientes($A$) son disjuntos, y de igual manera si $\lambda \in$ Primeros($\alpha$).

En el caso de nuestra gramática (sec. 4.2), se cumple la primera condición ya que la misma está factorizada a izquierda. En el caso de la segunda condición, hay que analizar las siguientes reglas de producción en las cuales $\lambda \in Primeros(\beta)$

$$
\begin{aligned}
< parte\ declaraciones\ subrutinas > ::= & < declaracion\ de\ subrutina >< mas\ subrutinas > \\
< mas\ subrutinas > ::= & \ ; < declaracion\ de\ subrutina >< mas\ subrutinas > \\
& \mid \lambda
\end{aligned}
$$

En este caso, como $\lambda \in Primeros(\lambda)$, entonces los $Primeros(\mathbf{;} < declaracion\ de\ subrutina >< mas\ subrutinas >)$ y $Siguientes(< mas\ subrutinas >)$ deben ser disjuntos. Luego:

$$
\begin{aligned}
Primeros(; < declaracion\ de\ subrutina >< mas\ subrutinas >) &= \{;\} \\
Siguientes(< mas\ subrutinas >) \in Siguientes(< parte\ declaraciones\ subrutinas >) &= \{begin\}
\end{aligned}
$$

Luego, al ser disjuntos, se cumple la segunda condición. Como este caso hay varios, siendo algunos de ellos: $< mas\ declaraciones >$, $< mas\ subrutinas >$, $< resto\ expresion\ simple >$ y $< resto\ termino >$, donde también se cumple la segunda condición para ser LL1. Pero existe un caso donde no se cumple, siendo este:

$$
\begin{aligned}
< \text{sentencia condicional} > ::= & \ if < \text{expresion} > then < \text{sentencia} >< \text{parte else} > \\
< \text{parte else} > ::= & \ else < \text{sentencia} > \\
& \mid \lambda
\end{aligned}
$$

Como $\lambda \in$ Primeros($\lambda$), verificamos que $Primeros(\mathbf{else} < sentencia >) \cap Siguientes(< parte\ else >) = \emptyset$

$$
\begin{aligned}
\text{Primeros}(else < sentencia >) &= \{else\} \\
\text{Siguientes}(< parte\ else >) &\subseteq Siguientes(< sentencia\ condicional >) \\
&\subseteq Siguientes(< sentencia >) \\
&\subseteq Primeros(< parte\ else >) = \{else\} \\
\text{Siguientes}(< parte\ else >) &\subseteq \{else\} \\
\{else\} \cap \{else\} &\neq \emptyset
\end{aligned}
$$

Luego, no se cumple LL(1) por el conflicto con **else**, llamado problema del "dangling else"(Aho y cols., 1986))

## 4.4. Analizador Sintáctico (pesudocodigo)

**Declaracion de Funcion**

```
void declaracion_de_funcion() {
    match(function);
    match(identificador);
    parametros_formales_opt();
    match(':');
    tipo();
    match(';');
    bloque();
}

void parametros_formales_opt() {
    if (preanalisis == '(') {
        match('(');
        lista_de_secciones();
        match(')');
    }
}

void lista_de_secciones() {
    seccion_de_parametros_formales();
    lista_de_secciones_p();
}

void lista_de_secciones_p() {
    if (preanalisis == ';') {
        match(';');
        seccion_de_parametros_formales();
        lista_de_secciones_p();
    }
}

void seccion_de_parametros_formales() {
    lista_de_identificadores();
    match(':');
    tipo();
}

void lista_de_identificadores() {
    match(identificador);
```

```
    lista_de_identificadores_p();
}

void lista_de_identificadores_p() {
    if (preanalisis == ',') {
        match(',');
        match(identificador);
        lista_de_identificadores_p();
    }
}

void tipo() {
    if (preanalisis == integer || preanalisis == boolean) {
        match(preanalisis);
    } else {
        reportar("error de sintaxis: esperado integer o boolean");
    }
}

void bloque() {
    parte_de_declaraciones_de_variables_opt();
    declaracion_de_subrutinas_opt();
    sentencia_compuesta();
}

void parte_de_declaraciones_de_variables_opt() {
    if (preanalisis == var) {
        match(var);
        lista_de_declaraciones();
        match(';');
    }
}

void lista_de_declaraciones() {
    declaracion_de_variables();
    lista_de_declaraciones_p();
}

void lista_de_declaraciones_p() {
    if (preanalisis == ';') {
        match(';');
        declaracion_de_variables();
```

```
        lista_de_declaraciones_p();
    }
}

void declaracion_de_variables() {
    lista_de_identificadores();
    match(':');
    tipo();
}

void declaracion_de_subrutinas_opt() {
    if (preanalisis == procedure || preanalisis == function) {
        declaracion_de_subrutinas();
    }
}

void sentencia_compuesta() {
    match(begin);
    lista_de_sentencias_opt();
    match(end);
}

void lista_de_sentencias_opt() {
    if (preanalisis != end) {
        sentencia();
        lista_de_sentencias_p();
    }
}

void lista_de_sentencias_p() {
    if (preanalisis == ';') {
        match(';');
        sentencia();
        lista_de_sentencias_p();
    }
}

void sentencia() {
    // Implementación específica omitida
}

void match(terminal t) {
```

```
    if (preanalisis == t) {
        preanalisis = siguienteTerminal;
    } else {
        reportar("error de sintaxis: esperado " + t);
    }
}
```

# 5. Trabajo de Laboratorio N5: Analizador Sintáctico

## 5.1. Descripción del Producto y Herramientas Utilizadas para su Desarrollo

El producto desarrollado es un analizador sintáctico descendente recursivo predictivo, diseñado para verificar si una secuencia de tokens (previamente generada por el analizador léxico definido en la sección 3) cumple con las reglas del subconjunto del lenguaje Pascal. La gramática ha sido previamente transformada para eliminar la recursión por la izquierda y adaptarse al enfoque descendente.

Para su desarrollo, se utilizó el lenguaje de programación Python debido a su flexibilidad y la facilidad para manejar cadenas de texto. En particular, se emplearon las siguientes herramientas y técnicas:

- **Python 3.x**: El lenguaje de programación principal utilizado para desarrollar el analizador sintáctico.
- **Editor de texto**: Se utilizó un editor de texto como Visual Studio Code para escribir y organizar el código.
- **Archivos de texto**: El programa toma como entrada un archivo de texto (.txt), lo lee y procesa su contenido.

## 5.2. Manual de Uso

Para ejecutar el analizador sintáctico, siga los siguientes pasos:

1. **Preparación del entorno de trabajo:**
   - Asegúrese de tener instalado Python 3.x en su sistema. Puede descargarlo desde el sitio oficial: `https://www.python.org/downloads/`.
2. **Cargar el archivo de entrada:**
   - Coloque el archivo de texto que contiene el código fuente que desea analizar en el mismo directorio que el programa o proporcione la ruta completa al archivo.
   - Asegúrese de que el archivo de entrada esté en formato .pas o .txt y contenga el código del lenguaje a analizar.
   - En la terminal, ejecute el script Python con uno de los siguientes comandos, según cómo esté configurado su sistema:

     ```
     python main.py entrada.txt
     ```

     o bien:

```
py main.py entrada.txt
```

- Asegúrese de estar ubicado en el mismo directorio que el archivo `main.py` y que el archivo `entrada.txt` exista.

3. **Ver los resultados:** Después de ejecutar el programa podrá observar en la terminal la respectiva salida.

## 5.3. Ejemplos de ejecuciones

**Entrada:**

```
program prueba;
var a b: integer;
begin
  a := 9;
end.
```

**Salida:**

```
Syntax error at line 2, column 12: expected 'asignacion_de_tipo', found 'ident'
```

Figura 6: Salida ejecución ejemplo con errores

**Entrada sin errores:**

```
program prueba;
var
  a,b:integer;
  c:boolean;
 begin
   read( b );
   c:= true;
   write( c )
 end.
```

**Salida:** Si no se encuentra ningún error, el programa retorna un mensaje de OK.

## 5.4. Problemas Encontrados

Uno de los desafíos encontrados fue la necesidad de adaptar el analizador léxico para que devolviera los tokens en el formato requerido por el analizador sintáctico. Esto implicó realizar ajustes en la forma en que se identifican y devuelven ciertos lexemas, como operadores y palabras reservadas, a fin de asegurar la compatibilidad entre ambas etapas del compilador.

Además, se identificó una dificultad relacionada con la interpretación de ciertos errores sintácticos. La figura 7 muestra un ejemplo de esta situación: el usuario escribe un ; luego de un end, lo cual, según la gramática, implica que deberían seguir más declaraciones de funciones o procedimientos. Sin embargo, se encuentra un begin, lo que genera un error, ya que no corresponde con lo que el analizador sintáctico espera en ese contexto. Este tipo de situaciones evidencia la necesidad de un sistema de reporte de errores más preciso, que no solo indique la línea y la columna en la que se produce el problema, sino que también brinde una descripción más clara y contextualizada del mismo. Como mejora futura, se propone implementar dicho sistema para facilitar el diagnóstico y la corrección de errores por parte del usuario.

```
program p1  ;
var
    a:integer ; b:boolean ;

procedure p1(a:integer) ;
var c:integer ;
begin
    a:= 1
end ;


begin
    a:= 3 ;
    b:= true ;
    if b then b(1)
end.
```

Figura 7: Código con error sintáctico

## 5.5. Conclusiones

Se logró desarrollar el analizador sintáctico en Python, tomando como base el pseudocódigo presentado en 4.4. Se describió su funcionamiento general y se incluyeron ejemplos de ejecución con el fin de ilustrar su salida. Asimismo, se expusieron las principales dificultades encontradas durante el desarrollo y se propusieron posibles mejoras para versiones futuras.

# 6. Trabajo Practico N6: Analizador Semántico

## 6.1. Tabla de Símbolos

Las tablas de símbolos (TS) son estructuras esenciales en los compiladores, ya que permiten almacenar y organizar la información asociada a los identificadores que aparecen en el programa fuente. Su propósito principal es asegurar la coherencia semántica durante la traducción del código a una representación ejecutable. Entre sus funciones más importantes se encuentran el registro de variables, constantes, funciones y otros elementos declarados; la resolución de referencias a identificadores durante el análisis, vinculando cada nombre con sus atributos (tipo de dato, dirección en memoria, ámbito, entre otros); y la detección de errores semánticos, tales como asignaciones incompatibles, el uso de variables no declaradas o la redefinición indebida de nombres dentro de un mismo bloque. Asimismo, la información que conserva la tabla resulta clave durante la fase de síntesis, ya que se aprovecha para la generación de código intermedio y finalmente de código ejecutable.

Por tratarse de un componente central, la tabla de símbolos debe diseñarse como un tipo de dato abstracto accesible a los diferentes módulos del compilador. Es indispensable que permita operaciones básicas como **inserción, consulta, actualización y eliminación de entradas** de manera eficiente. Para nuestra implementación, se eligió emplear una *tabla hash* como estructura de base. Un aspecto adicional a tener en cuenta es la existencia de distintos entornos o ámbitos en el programa (por ejemplo, bloques, funciones o procedimientos), lo cual habilita la posibilidad de que un mismo identificador sea declarado más de una vez en contextos válidos. Para manejar esta situación, existen dos alternativas: una tabla independiente por cada entorno organizada en una pila que mantiene siempre visible el ámbito actual o bien una única tabla global que asigne un número de entorno a cada declaración. En nuestro caso se adoptó esta primer estrategia: cada vez que se ingresa a un nuevo bloque se crea una tabla propia que guarda una referencia al entorno anterior, y las búsquedas de identificadores comienzan desde el tope de la pila hacia abajo.

Por otra parte, cada entorno implementa su tabla de símbolos utilizando una tabla hash, en la que la clave corresponde al lexema del identificador y el valor almacena la información asociada. Para representar a cada símbolo se definió una clase Symbol, que contiene su propia tabla hash destinada a registrar atributos como el nombre, el tipo de dato, la cantidad de parámetros en caso de funciones o procedimientos, y cualquier otro dato relevante para el análisis semántico. De este modo, la estructura no solo gestiona identificadores, sino que también conserva información detallada de cada uno de ellos, tal como puede observarse en un entorno donde se registre, por ejemplo, la definición de una función llamada resta junto con las variables asociadas. Esto se puede observar en la figura 8.

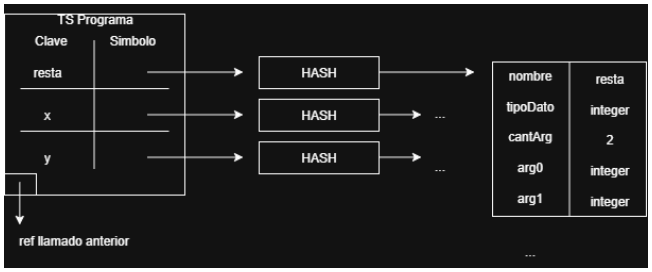


Figura 8: TS y clase symbol

A continuación se mostrará un ejemplo en el lenguaje Pascal.

```
program Example;

var
   x, y : integer;  { Ámbito global, Parse Point A }

procedure P;
var
   y, z : real;     { Ámbito local a P, Parse Point B }
begin
   y := 3.14;       { Se refiere al y de tipo real, no al global }
   z := y * 2;
end;

begin
   x := 5;
   y := 10;         { Se refiere al y global }
   P;
end.
```

**Parse Point A (ámbito global)**: Se crea una TS con x, y : integer.

**Parse Point B (dentro de P)**: Se abre un nuevo alcance con una TS que guarda y, z : real, y que referencia a la TS global. Cuando dentro de P se usa y, el compilador consulta primero la TS local y encuentra que y es real.

**Al cerrar el procedimiento**: La TS local se elimina, y el compilador vuelve al ámbito global. Por eso, al asignar y := 10 en el bloque principal, se usa el y global (de tipo integer).

## 6.2. Implementacion

### 6.2.1. La Tabla de Símbolos (`TablaSimbolos`)

La base del análisis semántico es la capacidad de registrar y consultar información sobre los identificadores (variables, funciones, etc.). Para ello, se creó la clase `TablaSimbolos`.

- **Estructura de Datos:** Internamente, utiliza un diccionario de Python (`self.tabla`) para almacenar la información de cada símbolo.
- **Gestión de Ámbito (Scope):** Se implementó un sistema de ámbito anidado para manejar variables locales y globales. La clase mantiene una pila de ámbitos (`self.ambitos`) y una referencia al ámbito actual (`self.ambito_actual`). Las funciones `entrar_ambito(nombre)` y `salir_ambito()` permiten al parser gestionar la entrada y salida de bloques de código como funciones y procedimientos.
- **Almacenamiento de Símbolos:** El método `insertar(...)` añade un nuevo símbolo. Para evitar colisiones y permitir el *shadowing*, la clave en el diccionario se construye combinando el ámbito y el nombre del símbolo (ej: `miFuncion.variableLocal`). Cada símbolo almacena: **nombre**, **tipo**, **categoría** (`variable`, `funcion`), **ámbito** y **parámetros**.
- **Búsqueda de Símbolos:** El método `buscar(nombre)` implementa la lógica de resolución de nombres. Inicia la búsqueda en el ámbito actual y, si no encuentra el símbolo, asciende recursivamente por los ámbitos padre hasta llegar al ámbito global.

### 6.2.2. El Analizador Semántico (`AnalizadorSemantico`)

Esta clase centraliza todas las reglas y verificaciones semánticas del lenguaje. Contiene una instancia de `TablaSimbolos` y ofrece una serie de métodos para validar diferentes construcciones del lenguaje.

**Métodos de Verificación Clave**

`verificar_declaracion(...)` Comprueba si un identificador ha sido declarado previamente llamando al método `buscar` de la tabla de símbolos.

`verificar_asignacion(...)` Valida que el tipo de la expresión a la derecha de una asignación (`:=`) sea compatible con el tipo de la variable a la izquierda.

`verificar_operacion_binaria(...)` Realiza la comprobación de tipos para operaciones aritméticas (`+, *`), relacionales (`=, <`) y lógicas (`and, or`).

`verificar_operacion_unaria(...)` Valida operaciones unarias como `not` (que solo aplica a booleanos) o el signo `-` (que solo aplica a enteros).

`verificar_llamada_funcion(...)` Se asegura de que una llamada a una función sea correcta, verificando la **aridad** (número de parámetros) y los **tipos de parámetros**.

### 6.2.3. Integración con el Analizador Sintáctico

La clave de la implementación fue ïnyectarÏlamadas a los métodos del `AnalizadorSemantico` en puntos estratégicos del parser. El objeto `semantico` se instancia globalmente para ser accesible desde todas las funciones de parsing.

**Puntos de Integración Notables**

1. **Declaraciones:** En `declaracion_de_variables`, después de reconocer la lista de identificadores, se llama a `semantico.tabla_simbolos.insertar()` para registrarlos. En las declaraciones de funciones, se inserta su firma en el ámbito global.

2. **Gestión de Ámbito:** Al inicio de `declaracion_de_funcion`, se llama a `semantico.tabla_simbolos.entrar_ambito()`. Al finalizar el bloque, se invoca a `semantico.tabla_simbolos.salir_ambito()`.

3. **Verificación de Expresiones:** En funciones como `resto_expresion_simple`, se capturan los tipos de los operandos y se invoca a `semantico.verificar_operacion_binaria()` para validar la operación.

4. **Validación de Sentencias:** En `sentencia_ident`, se distingue si es una asignación (validando con `verificar_asignacion()`) o una llamada a procedimiento. En `if` y `while`, se comprueba que la condición evaluada sea de tipo `boolean`.

### 6.2.4. Manejo de Errores Semánticos

Para el reporte de errores, los métodos del `AnalizadorSemantico` lanzan una excepción `SyntaxError`. Los mensajes de error son descriptivos e incluyen la línea y la columna donde ocurrió el fallo, junto con una explicación clara del problema (por ejemplo: "tipo `'string'` no es válido", "`'x'` no ha sido declarado").

# 7. Trabajo de Laboratorio N7: Generador de Código Intermedio (MEPA)

## 7.1. El Árbol de Sintaxis Abstracta (AST)

El **Árbol de Sintaxis Abstracta (AST)** constituye la **representación intermedia** canónica del programa fuente. Su propósito fundamental es servir como interfaz estructurada entre las fases de análisis y la fase de generación de código.

A diferencia del Árbol de Análisis Sintáctico (Parse Tree), el AST es **abstracto**, lo que implica que descarta detalles sintácticos que no son relevantes para la semántica o la generación de código, tales como paréntesis, punto y coma, y palabras clave puramente gramaticales. Esto asegura que la estructura refleje únicamente la lógica de la programación.

La implementación en `ast.py` define una jerarquía de nodos a partir de la clase base `NodoAST`, organizados en tres categorías principales:

### 7.1.1. Taxonomía de Nodos

**Nodos de Expresión** Estos nodos modelan los valores y cálculos que resultan de una evaluación. Son cruciales para la **desambiguación de la precedencia y asociatividad** de operadores:

- `NodoNumero` y `NodoBooleano`: Representan literales.
- `NodoIdentificador`: Referencia a una variable.
- `NodoOperacionBinaria`: Para operadores como `+`, `and`, `<`. La estructura jerárquica de este nodo (`izquierda`, `operador`, `derecha`) ya resuelve la precedencia.
- `NodoOperacionUnaria`: Para operadores como `-` (unario) o `not`.
- `NodoLlamadaFuncion`: Representa la invocación de una función.

**Nodos de Sentencia (Statement)** Estos nodos representan acciones y estructuras de control que alteran el estado del programa:

- `NodoAsignacion`: Define el flujo de datos (`variable := expresion`).
- `NodoIf` y `NodoWhile`: Encapsulan la lógica condicional y de bucles, respectivamente, y son los pilares para la generación de instrucciones de salto condicional (`DSVS`) en MEPA.
- `NodoRead` y `NodoWrite`: Encapsulan las operaciones de entrada/salida.

**Nodos de Estructura y Declaración** Definen la organización del programa y la creación de nuevos ámbitos léxicos:

- `NodoPrograma`: Nodo raíz que contiene las declaraciones de subrutinas y el bloque de código principal.
- `NodoDeclaracionProcedimiento` / `NodoDeclaracionFuncion`: Definen la firma (nombre y parámetros) y el cuerpo (`bloque_cuerpo`) de las subrutinas. Son esenciales para el precálculo de los **niveles léxicos** de MEPA.

### 7.1.2. Enriquecimiento Semántico del AST

Durante la fase de análisis semántico (en `sintactico.py` y `semantico.py`), el AST experimenta un proceso de **enriquecimiento**. A los nodos de expresión se les adjunta un atributo vital: el `tipo`.

- **Propagación de Tipos**: Para una `NodoOperacionBinaria`, el analizador semántico determina el `tipo` resultante (e.g., `integer` o `boolean`) basándose en las reglas de compatibilidad de tipos de los operandos.
- **Importancia para MEPA**: El generador de código MEPA se basa en este atributo `tipo` para emitir las instrucciones correctas. Por ejemplo, en Pascal, las operaciones booleanas y numéricas a menudo se resuelven de manera diferente, por lo que el tipo adjunto es una guía fundamental para la generación de código (e.g., usar instrucciones `SUMA` o `MULT` para `integer`, y `NEGA` o `O_LOGI` para `boolean`).

Al finalizar el análisis, el AST es una estructura **comprobada y tipificada** que simplifica drásticamente la tarea del `GeneradorMEPA`.

## 7.2. Diseño del Generador de Código MEPA

La clase `GeneradorMEPA` en `generador_mepa.py` es la encargada de traducir el AST y la información de la Tabla de Símbolos a instrucciones de la **Máquina de Ejecución de PASCAL (MEPA)**.

El proceso de generación se realiza en dos fases principales, un patrón común en compiladores para manejar el ámbito y el direccionamiento de memoria en lenguajes con anidamiento de subrutinas (como Pascal):

### 7.2.1. Fase 1: Pre-Cálculo de Nivel Léxico y Desplazamientos (Offsets)

Antes de generar cualquier instrucción, es fundamental calcular las direcciones de memoria relativas que la máquina MEPA necesita para acceder a las variables y parámetros.

1. **Nivel Léxico (`nivel`)**: Indica la profundidad de anidamiento de una subrutina. El ámbito global tiene nivel 0, y cada subrutina anidada incrementa el nivel. Se almacena en `self.info_niveles`.

2. **Desplazamiento (`offset`)**: Es la posición de la variable o parámetro relativa al inicio del marco de activación de su ámbito.

   - **Variables Locales**: Tienen `offset` positivos, comenzando en 0 y contando hacia arriba. Se utiliza la instrucción `RMEM` para reservar este espacio.
   - **Parámetros Formales**: Tienen `offset` negativos, ubicados antes del control link y el access link. Un parámetro $i$ en una subrutina con $n$ parámetros se direcciona en la posición $-(n + 3 - i)$.
   - **Valor de Retorno (Función)**: Se asigna un slot fijo en el marco de activación, generalmente en $-(n + 3)$, para que el cuerpo de la función asigne su resultado a este slot a través de la asignación `nombreFuncion := expresion`.

   Toda esta información de direccionamiento es calculada por `_precalculo_recursivo` y almacenada en `self.info_offsets` usando una clave única `ambito.nombre`.

### 7.2.2. Fase 2: Generación de Código (Visitor Pattern)

Una vez que todas las direcciones han sido resueltas, el método `_generar_recursivo` recorre el AST. El generador implementa el patrón **Visitor** dinámico: por cada nodo AST (`nodo.tipo_nodo`), llama al método `generar_tipo_nodo` correspondiente (e.g., `generar_asignacion`, `generar_operacion_binaria`).

#### Instrucciones Clave para Direccionamiento

- `APVL nivel, offset`: **Apilar Valor**. Carga el valor de una variable en la pila, utilizando su nivel léxico y offset. Utilizado para **expresiones** (`generar_identificador`).
- `ALVL nivel, offset`: **Almacenar Valor**. Saca un valor del tope de la pila y lo guarda en la variable. Utilizado para **asignaciones** (`generar_asignacion` y `generar_read`).

**Instrucciones de Subrutinas** El manejo de ámbitos se basa en las instrucciones de llamada y retorno:

- `ENPR nivel`: **Entrar Procedimiento**. Instrucción de prólogo de subrutina para configurar el nuevo marco de activación.
- `RTPR nivel, num_params`: **Retornar Procedimiento**. Instrucción de epílogo que limpia el marco de activación y salta al punto de retorno, ajustando la pila para remover los parámetros y el valor de retorno (si es función).
- `LLPR etiqueta`: **Llamar Procedimiento/Función**. Transfiere el control a la subrutina, cuyo código comienza en la `etiqueta` (el nombre de la subrutina).

**Tratamiento de Expresiones** La generación de expresiones (`generar_operacion_binaria`, `generar_operacion_unaria`) sigue el principio de **postfija**. Los operandos se evalúan y se apilan, y luego la instrucción del operador consume los operandos del tope y apila el resultado.

- **Ejemplo** (`A + B`)

```
APVL nivelA, offsetA
APVL nivelB, offsetB
SUMA
```

**Tratamiento de Llamadas a Función** Para una función, la instrucción `generar_llamada_funcion` precede la evaluación de parámetros con una instrucción `RMEM 1` para reservar el espacio en la pila donde la función depositará su valor de retorno.

## 7.3. Código de Inicialización del Programa

El método `generar_programa` define el flujo de ejecución principal:

1. `INPP` (Iniciar Programa).
2. `RMEM num_globales` (Reservar espacio para variables globales).
3. `DSVS etiqueta_main` (Saltar sobre las definiciones de subrutinas).
4. Se genera el código de todas las declaraciones de subrutinas.
5. `etiqueta_main NADA` (Punto de entrada al código principal).
6. Se genera el código del bloque principal.
7. `LMEM num_globales` (Liberar memoria global).
8. `PARA` (Parar la ejecución).

## 7.4. Manual de Uso

Para ejecutar el generador de codigo intermedio, siga los siguientes pasos:

1. **Preparación del entorno de trabajo:**
   - Asegúrese de tener instalado Python 3.x en su sistema. Puede descargarlo desde el sitio oficial: `https://www.python.org/downloads/`.

- Descomprima el archivo .zip que contiene el código fuente del compilador. En él encontrará el archivo **main.py** (programa principal), todas las etapas de la compilación (**lexico.py**, **sintactico.py**, etc.) y una carpeta llamada **pascal_test** con distintos ejemplos de prueba organizados en subcarpetas correspondientes a pruebas para cada etapa del compilador.

2. **Cargar el archivo de entrada:** Coloque el archivo de texto que contiene el código fuente que desea analizar en el mismo directorio que el programa o proporcione la ruta completa al archivo.

   - En la terminal, ejecute el script Python con uno de los siguientes comandos, según cómo esté configurado su sistema:

     ```
     python3 main.py <ruta al archivo .pas>
     ```

     o bien:

     ```
     py main.py <ruta al archivo .pas>
     ```

     Como por ejemplo:

     ```
     python3 main.py ./pascal_test/mepa/factorial.pas
     ```

   - Asegúrese de estar ubicado en el mismo directorio que el archivo `main.py` y que el archivo de entrada exista.

3. **Resultado:** Después de ejecutar el programa, podrá observar en la terminal la salida correspondiente, la cual puede incluir avisos de errores léxicos, sintácticos o semánticos. En caso de que todo sea correcto, se mostrará un mensaje de éxito indicando la ubicación del archivo .mepa, que será generado con el mismo nombre que el archivo .pas y se almacenará en el mismo directorio.

# Referencias

Aho, A. V., Sethi, R., y Ullman, J. D. (1986). *Compilers: Principles, techniques, and tools.* Addison-Wesley.

Kowaltowski, T. (1983). *Implementação de linguagens de programação.* Guanabara Dois.